## 🔥 Internal Working of HashMap (Java 8 and above)

A **HashMap** stores key-value pairs (Entry objects) internally using:

Array + LinkedList + Balanced Tree (Red-Black Tree)

Structure:

Node<K,V>[] table  (an array of buckets)

Each bucket → LinkedList or Tree of nodes

---

## 1️⃣ How HashMap Stores a Key-Value Pair

When you call:

**map.put(key, value);**

**Steps inside:**

**Step 1: Compute hash**

HashMap calls:

hash = hash(key.hashCode());

This improves hash distribution using bit manipulation.

**Step 2: Calculate bucket index**

index = (n - 1) & hash // n = table size

Index ensures the value falls within array bounds.

**Step 3: Insert into bucket**

Case 1: **Empty bucket**

table[index] = new Node(key, value)

Case 2: **Same key already exists**

- HashMap checks:
  - hash equality
  - then equals() on keys
- If same key, **value is updated**, not added.

Case 3: **Collision** (different key → same index)

- New node is appended to bucket

- Bucket is **LinkedList** initially

Example:

index 5 → Node1 → Node2 → Node3

---

### 2️⃣ Collision Handling (Very Important)

**Java 7:**

Collisions always used **LinkedList** → O(n) worst-case lookup
Risk of **hash collision attack** → slow.

**Java 8+:**

If a bucket becomes too large:

- When LinkedList size **> 8** → **Convert to Red-Black Tree**
  (Called *treeification*)

This improves lookup:

| Structure | Worst Case Lookup |
|---|---|
| LinkedList | O(n) |
| Red-Black Tree | O(log n) |

---

### 3️⃣ Treeification Threshold

Treeification happens only when:

Bucket size > 8

AND

array capacity >= 64

If capacity < 64, HashMap **expands array** instead of forming tree.

---

### 4️⃣ Resizing / Rehashing

Default capacity = **16**
Load factor = **0.75**

If size exceeds threshold:

threshold = capacity * loadFactor

Example:

16 * 0.75 = 12

→ When 12 entries inserted → resize triggered

**Resizing Steps:**

1. New array created → size = old * 2
2. All nodes rehashed
3. Nodes redistributed to new positions

Java 8 optimization:

- During resize, a node either stays at the same index
  OR moves to **index + oldCapacity**

This avoids recomputing full hash.

---

### 5️⃣ How HashMap Retrieves Data (get())

When you call:

**map.get(key);**

Steps:

1. Compute hash → find index
2. Go to bucket
3. If first node matches hash + key → return value
4. Else:
    - If bucket is a **tree**, search in tree (O(log n))
    - If bucket is a **list**, search list (O(n))

---

### 6️⃣ Null Keys and Null Values

HashMap allows:

- **One null key**
- **Multiple null values**
- **HashTable does not allow null keys or null values** because it uses synchronized operations and nulls would break its internal logic.

Null key always stored at **index 0**.

---

### 7️⃣ HashMap is NOT Thread-Safe

Multiple threads modifying HashMap can cause:

- Lost updates
- Infinite loops (pre-Java 8)
- Data corruption

For thread safety:

- Use ConcurrentHashMap or HashTable
- Or wrap with Collections.synchronizedMap()

---

### 8️⃣ Important Interview Points

#### ⭐ Why tree structure?

To avoid O(n) worst-case caused by hash collision attacks.

#### ⭐ Why threshold 8 for treeification?

Empirically chosen:
Probability of >8 collisions is extremely low unless intentionally attacked.

#### ⭐ Why require minimum capacity 64?

To avoid converting to tree too early; resizing often solves collisions.

---

### 📝 Short & Sweet Summary

| Component | Purpose |
|---|---|
| Array | Buckets |
| LinkedList | Collision chain (few items) |
| Red-Black Tree | High collision buckets (many items) |
| Load Factor | Controls resizing |
| Hash function | Spreads keys |